상태 페이지

docker-compose.yml

```yaml
# ─────────────────────────────────────────────────────────────────────────────
# optime-home 통합 개발 환경 (Docker Compose)
# ─────────────────────────────────────────────────────────────────────────────
services:
  optime-home:
    image: local/optime-home:1
    container_name: optime-home
    restart: unless-stopped
    # 호스트 네트워크 모드: 컨테이너가 호스트의 모든 포트를 직접 사용
    network_mode: host
    # 환경 변수 파일
    env_file:
      - .env
    # 헬스체크 설정
    healthcheck:
      test: ["CMD", "curl", "-f", "http://127.0.0.1:3001/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
    # 로그 설정
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    # 볼륨 마운트
    volumes:
      - optime_home_data:/app/data
    # 의존성 없음 (단일 컨테이너)
    depends_on:
      - auth-service
      - user-service
      - gateway
```

User-service/health.controller

```
import { Controller
import { ApiTags, ApiOperation
import { HealthCheckService,
  HttpHealthIndicator,
  MemoryHealthIndicator,
  DiskHealthIndicator, HealthCheckResult
} from '@nestjs/terminus';

@ApiTags('health')
@Controller('health')
export class HealthController {
  constructor(
    private readonly health: HealthCheckService,
    private readonly http: HttpHealthIndicator,
    private readonly memory: MemoryHealthIndicator,
    private readonly disk: DiskHealthIndicator,
  )
  {}

  @Get()
  @HealthCheck()
  @ApiOperation({
    summary: '서비스 헬스체크',
  })
  check(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }

  // 상세 헬스체크 (DB 연결 등 확인)
  @Get('deep')
  @HealthCheck()
  @ApiOperation({
    summary: '상세 헬스체크 (DB 연결 등 확인)',
  })
  deepCheck(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }
}
```

Gateway/health.controller

```
import { Controller, Get, Logger, Inject
import { ClientProxy, MicroservicePattern
import { firstValueFrom, timeout, catch
import { ApiTags, ApiOperation, ApiResponse
import { HealthCheckService, HttpHealthIndicator,
  MemoryHealthIndicator, DiskHealthIndicator,
  HealthCheckResult
} from '@nestjs/terminus';

@ApiTags('health')
@Controller('health')
export class HealthController {
  private readonly logger = new Logger(HealthController
  private readonly health: HealthCheckService,
  private readonly http: HttpHealthIndicator,
  private readonly memory: MemoryHealthIndicator,
  private readonly disk: DiskHealthIndicator,
  private readonly authClient: ClientProxy,
  private readonly userClient: ClientProxy,
  )
  {}

  // 기본 헬스체크 (빠르고 가벼움 — 로드밸런서 probe 용)
  @Get()
  @HealthCheck()
  @ApiOperation({
    summary: '서비스 헬스체크',
  })
  check(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }

  // 상세 헬스체크 (DB 연결 등 확인)
  @Get('deep')
  @HealthCheck()
  @ApiOperation({
    summary: '상세 헬스체크 (DB 연결 등 확인)',
  })
  deepCheck(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }

  // 마이크로서비스 핑 테스트
  @Get('ping')
  @ApiOperation({
    summary: '마이크로서비스 핑 테스트',
  })
  ping(): Promise<{ [key: string]: 'ok' | 'error'
  }
  {
    const [authResult, userResult] = [
      this.pingService(this.authClient, 'auth-service
      this.pingService(this.userClient, 'user-service
    ];
    return {
      'auth-service': authResult,
      'user-service': userResult,
    }
  }
}
```

Auth-service/health.controller

```
import { Controller, Get, Logger, Inject
import { ClientProxy, MicroservicePattern
import { firstValueFrom, timeout, catch
import { ApiTags, ApiOperation, ApiResponse
import { HealthCheckService, HttpHealthIndicator,
  MemoryHealthIndicator, DiskHealthIndicator,
  HealthCheckResult
} from '@nestjs/terminus';

@ApiTags('health')
@Controller('health')
export class HealthController {
  private readonly logger = new Logger(HealthController
  private readonly health: HealthCheckService,
  private readonly http: HttpHealthIndicator,
  private readonly memory: MemoryHealthIndicator,
  private readonly disk: DiskHealthIndicator,
  )
  {}

  // 기본 헬스체크 (빠르고 가벼움 — 로드밸런서 probe 용)
  @Get()
  @HealthCheck()
  @ApiOperation({
    summary: '서비스 헬스체크',
  })
  check(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }

  // 상세 헬스체크 (DB 연결 등 확인)
  @Get('deep')
  @HealthCheck()
  @ApiOperation({
    summary: '상세 헬스체크 (DB 연결 등 확인)',
  })
  deepCheck(): Promise<HealthCheckResult> {
    return this.health.check([]);
  }

  // 마이크로서비스 핑 테스트
  @Get('ping')
  @ApiOperation({
    summary: '마이크로서비스 핑 테스트',
  })
  ping(): Promise<{ [key: string]: 'ok' | 'error'
  }
  {
    const [authResult, userResult] = [
      this.pingService(this.authClient, 'auth-service
      this.pingService(this.userClient, 'user-service
    ];
    return {
      'auth-service': authResult,
      'user-service': userResult,
    }
  }
}
```

```
http://localhost:3001/health/deep
GET
Header: Accept: application/vnd.nestjs.health+
Response: 200 OK
{
  "status": "ok",
  "info": {
    "database": { "status": "up" },
    "memory_heap": { "status": "up", "mb": 42.
    "memory_rss": { "status": "up", "mb": 68.
    "disk_storage": { "status": "up", "path":
    "auth-service": { "status": "up", "latency
    "user-service": { "status": "up", "latency
  },
  "error": {},
  "details": {
    "database": { "status": "up" },
    "memory_heap": { "status": "up", "mb": 42.
    "memory_rss": { "status": "up", "mb": 68.
    "disk_storage": { "status": "up", "path":
    "auth-service": { "status": "up", "latency
    "user-service": { "status": "up", "latency
  }
}
```

@healthCheck

```
{
  "status": "ok" | "error",
  "info":   { /* 통과한 체크들 */ },
  "error":  { /* 실패한 체크들 */ },
  "details": { /* 모든 체크 (info + error 합친 것) */ }
}
```

뒤적 서비스 다른

시

frontend/src/hooks/login.tsx

```
const [ ] = useAuthStore();
const [ ] = useUserStore();

const [ ] = [ ];

const [ ] = [ ];

const [ ] = [ ];
```